

Notes on the Problem 5 Backtest

buhl.pairs_backtest, prob5_driver.py, and how trades are tracked

Dante Buhl

October 2026

Summary

- **New function buhl.pairs_backtest:** trades one pair's standardized spread z_t under the rule in 5(a). Your lag-day trend check is an optional entry filter.
- **prob5_driver.py, lines 1–106, rewritten:**
 - reads the formation-period results (`sector_pairs.dat`, `kalman_data.dat`);
 - downloads the test period once (2024-01-01 to 2026-10-01);
 - builds the static and Kalman z -scores without lookahead;
 - runs four variants per pair: static/Kalman spread, each with the trend filter off and on;
 - records every trade in an inventory and a ledger.
- **Trade tracking:** an *inventory* (shares held at every close) and a *ledger* of every purchase and sale. Sharpe ratio, drawdown, turnover, trade counts and transaction costs can all be computed from these two afterwards without rerunning the backtest (Section 3).

1 Changes to prob5_driver.py

Problems in the original shell.

Problem	Fix
<code>pairs_filename</code> , <code>sector_tickers</code> , <code>inv</code> undefined	one <code>data_dir</code> ; tickers from the pairs file; inventory arrays (Section 3)
<code>set(ticker_a+ticker_b)</code> joins strings element by element	<code>sorted(set(ticker_a) set(ticker_b))</code>
prices logged twice (<code>np.log</code> of log prices)	separate <code>prices</code> and <code>log_prices</code> dictionaries of arrays
<code>fit.SDE</code> and <code>kalman_spread</code> refit on the test data (lookahead)	all parameters come from the formation-period files
<code>mu</code> , <code>theta</code> , <code>sigma</code> unpacked in the wrong order	no refit is needed, so the call is gone
entry threshold used the OU σ	uses the stationary std of the spread, <code>stat_std_dev</code>
α missing from the static spread	$S_t = \ln P_t^A - \alpha - \beta \ln P_t^B$
<code>db.BFD(y, j+1, dt)</code> : shape mismatch for $j \geq \text{lag}$	<code>BFD(1P[t-lag:t+1], lag+1, dt)</code> , i.e. $n = \text{len}(y)$
<code>else if, inv & ...</code>	valid Python

Data in. The two formation-period files have one row per pair in the same order. The driver checks this and stops with an error if the tickers don't match (rerun `prob4_driver.py` after any new sweep). The test prices are downloaded in one call and stored as dictionaries of numpy arrays on a shared date grid, the same pattern as Problem 4:

```
tickers = sorted(set(ticker_a) | set(ticker_b))
ticker_prices = yf.download(tickers, start=start, end=end, auto_adjust=True,
                           progress=False)["Close"]
dates = ticker_prices.index
prices = {ticker: np.array(ticker_prices[ticker]) for ticker in tickers}
log_prices = {ticker: np.log(prices[ticker]) for ticker in tickers}
```

Spreads without lookahead. For each pair, keep only the days where both prices exist (`mask`). Then build:

$$z_t^{\text{static}} = \frac{\ln P_t^A - \alpha - \beta \ln P_t^B - \mu}{\sigma_{\text{stat}}}, \quad z_t^{\text{kalman}} = \frac{e_t - \mu_k}{\sigma_{\text{stat},k}},$$

where all of $(\alpha, \beta, \mu, \sigma_{\text{stat}})$ come from `sector_pairs.dat`. The Kalman spread e_t continues the filter from its state on the last formation day, with $(\mu_k, \sigma_{\text{stat},k})$ from `kalman_data.dat`. e_t is the one-step prediction error, so it uses only $(\alpha_{t-1}, \beta_{t-1})$ and the prices at t .

```
alpha_k, hr_k, spread_k, P_k, R_k, lPb_mean_k = db.kalman_spread(lPa, lPb,
    dt, kalman["sigma_alpha"][i], kalman["sigma_beta"][i],
    alpha0=kalman["alpha"][i], beta0=kalman["hedge_ratio"][i], P0=P0,
    R=kalman["R"][i], lPb_mean=kalman["lPb_mean"][i])
z_kalman = (spread_k - kalman["mu"][i])/kalman["stat_std_dev"][i]
```

2 The trading rule: `buhl.pairs_backtest`

```
def pairs_backtest(Pa, Pb, lPa, lPb, z, hedge_ratio, z_in=2., z_out=1.,
    lag=10, dt=1./252, trend_filter=False):
```

The position state is $s \in \{-1, 0, +1\}$ (short spread, flat, long spread). The decision on day t uses z_t :

State	Condition at close t	Action at close $t + 1$
flat	$z_t > z_{\text{in}}$	$s = -1$: sell \$1 of A, buy β_t of B
flat	$z_t < -z_{\text{in}}$	$s = +1$: buy \$1 of A, sell β_t of B
in a trade	$ z_t < z_{\text{out}}$	sell back the shares held, $s = 0$
in a trade	last valid day	forced close

The defaults are $z_{\text{in}} = 2$ and $z_{\text{out}} = 1$, from your comments.

Why these positions. A long spread position pays $\Delta \ln P^A - \beta \Delta \ln P^B = \Delta S$. When z is very negative, the spread is expected to rise back toward μ , so we go long, and the reverse when z is very positive. \$1 of A against β of B is the dollar hedge that matches the hedge ratio, since β is defined in log prices.

No lookahead. The signal is computed from the close on day t , and the trade fills at the close on day $t + 1$. No entry is allowed whose fill would land on the last day. The first `lag` days are skipped so the trend filter always has a full window.

Sizing. The share counts are set at entry and held until exit:

$$n_A = \frac{s}{P_{t+1}^A}, \quad n_B = -\frac{s \beta_t}{P_{t+1}^B}.$$

There is no daily re-hedging. For the Kalman variants, β_t moves every day, but it only enters through the z -score and the hedge at entry. That keeps turnover (and costs) down. Daily re-hedging would be a different strategy.

Trend filter. With `trend_filter=True`, an entry is only taken if the leg being bought has fallen over the last `lag` days, using your backward difference:

```
lP_buy = lPa if (s == 1) else lPb
trend = np.sum(BFD(lP_buy[t-lag:t+1], lag+1, dt))
if (trend >= 0):
    s = 0
```

The sum of the backward differences telescopes to $(\ln P_t - \ln P_{t-\text{lag}})/\Delta t$, the log return over the window. Exits are never filtered, so a position is never kept open because of the trend.

3 How trades are tracked

Two records are kept, following your inventory plus purchases/sales idea.

1. Inventory (the inv). For each variant `v`, `shares_a[v]` and `shares_b[v]` are arrays of shape (number of pairs $\times$ test days). Entry $[i, t]$ is the number of shares of that leg held by pair i at the close of day t . Positive is long and negative is short.

```
variants = [('static', False), ('static', True), ('kalman', False),
            ('kalman', True)]
shares_a = {v: np.zeros((num_pairs, nt_all)) for v in variants}
shares_b = {v: np.zeros((num_pairs, nt_all)) for v in variants}
```

Because holdings are stored for every day, the daily P&L is a single product:

$$\text{PnL}_t = \sum_i n_{i,t-1}^A (P_{i,t}^A - P_{i,t-1}^A) + n_{i,t-1}^B (P_{i,t}^B - P_{i,t-1}^B).$$

Missing days. The backtest runs only on a pair's valid days, so results are mapped back to the full date grid. A missing price in the middle of the window must not close the position, so each missing day takes the holdings of the last valid day:

```
full_range = np.arange(days[0], days[-1]+1)
last_valid = np.searchsorted(days, full_range, side='right') - 1
shares_a[v][i, full_range] = sa[last_valid]
```

When computing P&L, also forward-fill prices over missing days. This fixed a real mismatch: LBRDA has a gap partway through the test period.

2. Ledger (purchases and sales). Every transaction leg is one row of a numpy structured array:

field	meaning
pair	row index into the pairs files
day	index into dates (the fill day)
leg, ticker	'A' or 'B' and its ticker
shares	signed: > 0 purchase, < 0 sale
price	fill price
branch, trend_filter	which variant made the trade

```
purchases = ledger[ledger['shares'] > 0]
sales = ledger[ledger['shares'] < 0]
```

A round trip is four rows: the entry and the exit of both legs.

What each metric needs.

Quantity	From
daily P&L, Sharpe, drawdown	inventory and forward-filled prices (formula above)
number of trades	ledger rows / 4, per variant
turnover	$\sum \text{shares} \cdot \text{price} $ over the ledger
transaction costs	$c \text{shares} \cdot \text{price} $ subtracted on each row's day
cost sweep (5b)	repeat the line above for several c ; no new backtest

For example, net daily P&L at cost rate c for variant v :

```
in_v = (ledger['branch'] == v[0]) & (ledger['trend_filter'] == v[1])
cost = np.zeros(nt_all)
np.add.at(cost, ledger['day'][in_v],
          c*np.abs(ledger['shares'][in_v]*ledger['price'][in_v]))
pnl_net = pnl_gross - cost
```

4 Verification

Synthetic path (40 days, $\beta = 1.3$):

- a $z = 2.5$ signal on day 12 fills on day 13: short 0.0099 shares of A at \$101 (\$1) and long 0.0265 shares of B at \$49 (\$1.30). The exit signal on day 18 fills on day 19;
- a long entry that never meets the exit condition is closed on the last day;
- the trend filter blocks a long-spread entry while A is rising, and allows a short-spread entry while B is falling;
- a signal on the second-to-last day doesn't open a trade;
- each leg's shares sum to zero.

Full run (931 pairs, 689 test days):

- every (pair, leg, variant) nets to zero shares, and nothing is held on the last day;
- the total Kalman P&L summed from the inventory equals the ledger's cash flows $-\sum \mathbf{shares} \cdot \mathbf{price}$ exactly (-145.263883 both ways, before costs, at \$1 notional per trade).

Variant	trades	ledger rows
static	1966	7864
static + trend filter	1880	7520
Kalman	4345	17380
Kalman + trend filter	3909	15636

Still to do (after line 106): Sharpe ratio, maximum drawdown, turnover and trade counts before and after costs; the cost sweep; and the broken-pair diagnostic for 5(c).